Canary Release: Despliegue Progresivo con Nginx, Docker Compose y GitHub Actions

Christian Guillermo Rosas Hernández

1. Fundamentos: Por qué Surge Canary Release

En la ingeniería de software moderna y las operaciones de entrega continua (DevOps), la liberación de nuevas versiones a producción conlleva un dilema inherente entre velocidad de entrega y mitigación del riesgo. Históricamente, las estrategias de despliegue han evolucionado para resolver problemas críticos de disponibilidad y confiabilidad:

• Despliegue Recreate (Tradicional): La versión antigua (v1) se destruye y detiene antes de iniciar la nueva (v2). Esto provoca una interrupción obligatoria del servicio (downtime) y, si la nueva versión falla, la reversión requiere volver a desplegar la anterior, prolongando la caída.
• Despliegue Blue-Green: Se mantienen dos entornos de producción idénticos (Blue y Green). Aunque elimina el downtime cambiando el tráfico instantáneamente a nivel de balanceador, introduce un riesgo crítico: la conmutación es de tipo 'todo o nada' (0% a 100%). Si v2 contiene un defecto no detectado en las pruebas sintéticas, la totalidad de los usuarios reales quedará expuesta al error de forma inmediata.

Para solucionar esta debilidad surge Canary Release. Esta estrategia reduce al mínimo el radio de impacto inicial (Blast Radius): la nueva versión (Green / v2) se libera inicialmente a una porción muy pequeña del tráfico real (por ejemplo, un 10%). Mientras el 90% de los usuarios continúa en la versión consolidada y probada (Blue / v1), los sistemas de observabilidad monitorizan el comportamiento de v2 bajo condiciones reales de carga. Si la versión cumple con los criterios de aceptación y los SLOs (latencia baja, cero errores 5xx, transacciones completadas), se incrementa gradualmente la exposición (10% → 50% → 100%). Si se detecta una anomalía, el tráfico hacia el canario se cancela al instante (Rollback a 0%), protegiendo a la gran mayoría de los usuarios de experimentar el fallo.

Escenarios donde Canary resulta apropiado:
1. Servicios web con suficiente volumen de tráfico para obtener muestras estadísticamente significativas en ventanas de tiempo razonables.
2. Cambios en la lógica de negocio de riesgo moderado o alto, donde es indispensable cotejar tasas de error y latencia en producción.
3. Equipos con observabilidad madura (métricas, logs, tracing) y capacidad de reversión instantánea automatizada.
4. APIs y microservicios diseñados con contratos compatibles hacia atrás y tolerancia a versiones concurrentes.

Contraindicaciones (Cuándo NO usar Canary por sí solo):
• Modificaciones irreversibles o incompatibles en la base de datos sin aplicar antes el patrón Expand and

Contract.
• Operaciones con efectos secundarios externos irreversibles (p. ej., cobros bancarios definitivos o llamadas a pasarelas sin idempotencia).
• Aplicaciones monolíticas con muy bajo volumen de tráfico donde una muestra del 10% tarda días en recolectar evidencia.
• Sesiones de usuario altamente acopladas a la memoria del proceso sin afinidad (sticky sessions) ni almacenamiento distribuido de estado.

Tabla 1: Comparativa Técnica entre Estrategias de Despliegue

Criterio	Recreate (Tradicional)	Blue-Green	Canary Release
Disponibilidad (Downtime)	Existe ventana de caída obligatoria durante el reemplazo.	Cero tiempo de caída (conmutación atómica en proxy).	Cero tiempo de caída; coexistencia de versiones en producción.
Radio de Impacto (Blast Radius)	100% de los usuarios afectados ante fallos de la nueva versión.	100% de los usuarios expuestos al conmutar el balanceador.	Mínimo y acotado (inicialmente 5% a 10% del tráfico).
Velocidad de Rollback	Lenta: requiere re-desplegar o reconstruir v1.	Inmediata: conmutar el proxy de vuelta a Blue.	Inmediata: restaurar ponderación 100/0 hacia Blue en milisegundos.
Costo de Infraestructura	Bajo: solo un ambiente activo a la vez.	Alto: requiere duplicar el 100% de capacidad simultánea.	Flexible: puede usar réplicas ajustadas según el peso porcentual.
Complejidad de Observabilidad	Básica: verificación binaria de 'arriba o abajo'.	Media: validación pre-release en staging/Green.	Alta: requiere telemetría en tiempo real, SLIs y análisis estadístico.

2. Demostración Corta: Nginx Ponderado (Weighted Upstream)

Para implementar el enrutamiento porcentual en este laboratorio, Nginx actúa como proxy inverso y balanceador de carga HTTP. Mediante la directiva 'upstream' y el parámetro 'weight', Nginx implementa de forma nativa el algoritmo Weighted Round-Robin:

```
upstream backend {
    server blue:3000 weight=9;   # 90% del peso asignado a Blue (v1)
    server green:3000 weight=1;  # 10% del peso asignado a Green (v2)
}
server {
    listen 80;
    location / {
        proxy_pass http://backend;
```

```
        proxy_http_version 1.1;
        proxy_set_header Connection "close";
        proxy_set_header Host $host;
    }
}
```

Diferencia Clave: Proporción de Solicitudes vs. Usuarios Únicos

Principio de Enrutamiento Ponderado: El peso 9:1 en Nginx expresa una proporción aproximada de distribución sobre solicitudes HTTP individuales, no una segmentación determinista ni una asignación persistente del 10% de usuarios únicos. En una ráfaga pequeña de 10 peticiones, la naturaleza del round-robin distribuirá exactamente 9 a blue y 1 a green si las peticiones son secuenciales; sin embargo, en un entorno con conexiones concurrentes, reintentos o keep-alive, el conteo en muestras pequeñas varía y busca una convergencia estadística hacia el 90/10 conforme aumenta N (Ley de los Grandes Números).

3. Laboratorio: Arquitectura, Despliegue y Resultados

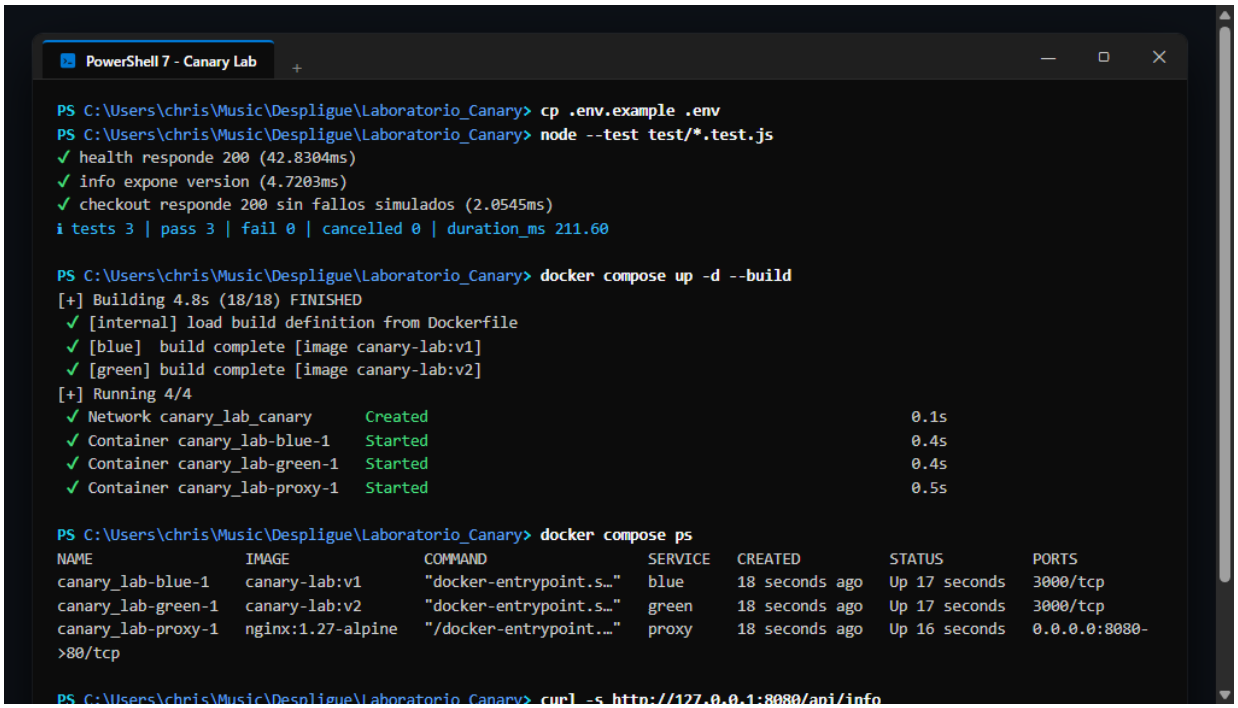
La arquitectura del laboratorio se compone de tres microservicios orquestados mediante Docker Compose:
1. blue: Contenedor Node.js compilado con RELEASE_VERSION=v1 que ejecuta la API en el puerto 3000 dentro de la red interna.
2. green: Contenedor Node.js compilado con RELEASE_VERSION=v2 que expone la nueva versión en el puerto 3000.
3. proxy: Servidor Nginx (nginx:1.27-alpine) que expone el puerto 8080 hacia el host y enruta el tráfico hacia blue y green mediante un archivo montado por bind (nginx/active.conf).

La aplicación API Node.js ofrece tres endpoints:
• /health: Verificación básica de salud del servicio (liveness). Siempre responde HTTP 200 OK en ambas versiones.
• /api/info: Expone la versión activa del contenedor que atendió la petición ('v1' o 'v2') y cabecera 'X-App-Version'.
• /api/checkout: Endpoint transaccional crítico que acepta compras (200 OK) en condiciones normales, pero devuelve 503 Service Unavailable si SIMULATE_ERROR=true en v2.

3.1 Preparación y Levantamiento del Entorno

Se procedió con la configuración del archivo '.env' a partir de '.env.example', la ejecución de la suite de pruebas unitarias en Node.js ('node --test test/*.test.js') donde los tres tests pasaron exitosamente (salud, info y checkout), y la construcción e inicio simultáneo de los contenedores mediante 'docker compose up -d --build'.

Evidencia Gráfica: Ejecución de Pruebas Unitarias y Docker Compose Up / PS

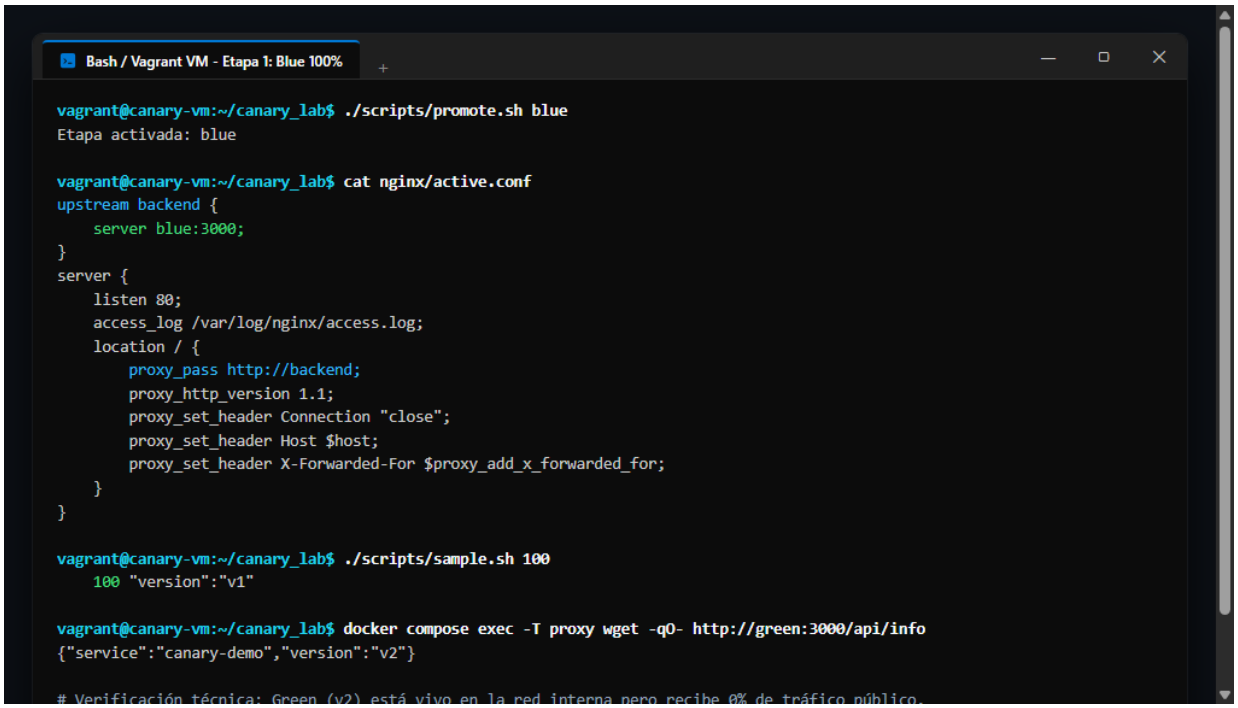


Descripción técnica: Salida de consola donde se observa la aprobación de las 3 pruebas unitarias de Node.js, la creación de la red 'canary', la compilación de las imágenes 'canary-lab:v1' y 'canary-lab:v2', el estado 'Up' de los 3 contenedores y la respuesta inicial de /api/info en v1.

3.2 Etapa 1: Línea Base (Blue 100%)

Se aplicó la etapa inicial con './scripts/promote.sh blue'. El script copió 'nginx/stages/blue.conf' hacia 'nginx/active.conf', validó la sintaxis con 'nginx -t' y recargó Nginx en caliente con 'nginx -s reload'. Al inspeccionar el archivo activo, el bloque upstream contiene únicamente 'server blue:3000;'.
Al ejecutar el script de muestreo de 100 peticiones ('./scripts/sample.sh 100'), el 100% de las respuestas provino de la versión 'v1'. Se confirmó además que Green (v2) está vivo en la red interna de Compose (respondiendo a consultas wget internas), pero completamente aislado del tráfico de usuarios.

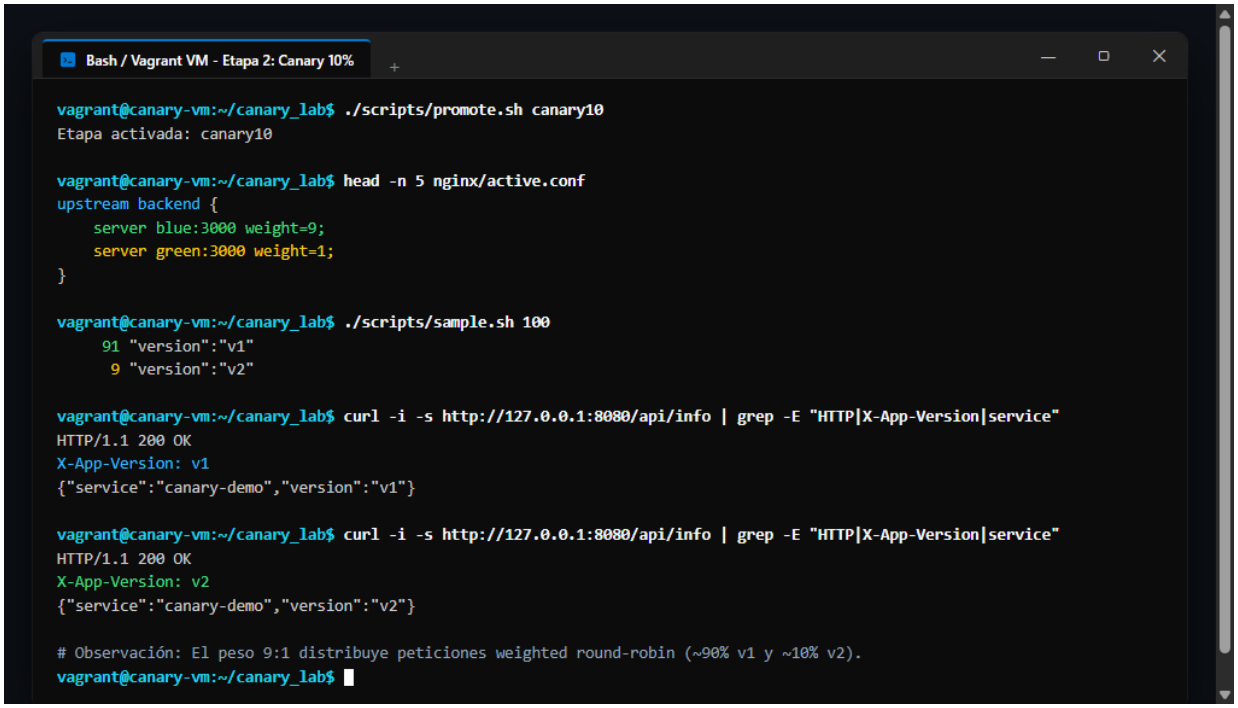
Evidencia Gráfica: Etapa 1: Línea Base con 100% de Tráfico en Blue (v1)



3.3 Etapa 2: Canary al 10% (Ponderación 9:1)

Se ejecutó './scripts/promote.sh canary10' para activar la configuración con 'weight=9' para Blue y 'weight=1' para Green. Al ejecutar el muestreo de 100 peticiones con './scripts/sample.sh 100', se obtuvieron 91 respuestas de 'v1' y 9 respuestas de 'v2'. Adicionalmente, se comprobaron las cabeceras HTTP mediante curl, observando 'X-App-Version: v1' y 'X-App-Version: v2' según el balanceo de carga.

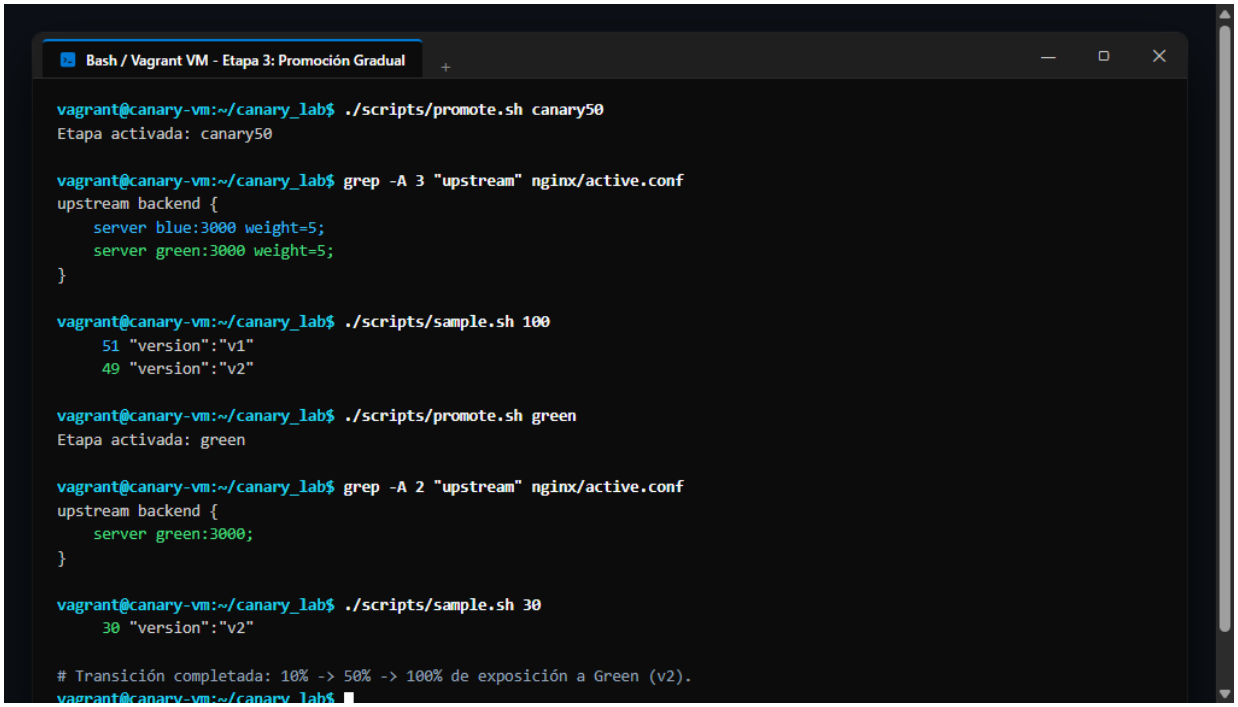
Evidencia Gráfica: Etapa 2: Canary al 10% con Conteo Ponderado 91/9



3.4 Etapa 3: Promoción Gradual (50% y Consolidación 100% Green)

Tras verificar la estabilidad en el 10%, se procedió al incremento gradual a 50% con './scripts/promote.sh canary50'. La configuración asignó 'weight=5' para Blue y 'weight=5' para Green. El muestreo de 100 solicitudes reflejó una distribución casi simétrica: 51 peticiones atendidas por 'v1' y 49 por 'v2'. Finalmente, tras confirmar que el 50% operó sin anomalías, se realizó la promoción total a producción con './scripts/promote.sh green'. Nginx pasó a enrutar exclusivamente hacia 'server green:3000;'. Un muestreo de 30 solicitudes arrojó 30 respuestas de 'v2' (100% Green), completando satisfactoriamente la transición de versión.

Evidencia Gráfica: Etapa 3: Promoción a Canary 50% y Consolidación en Green 100%



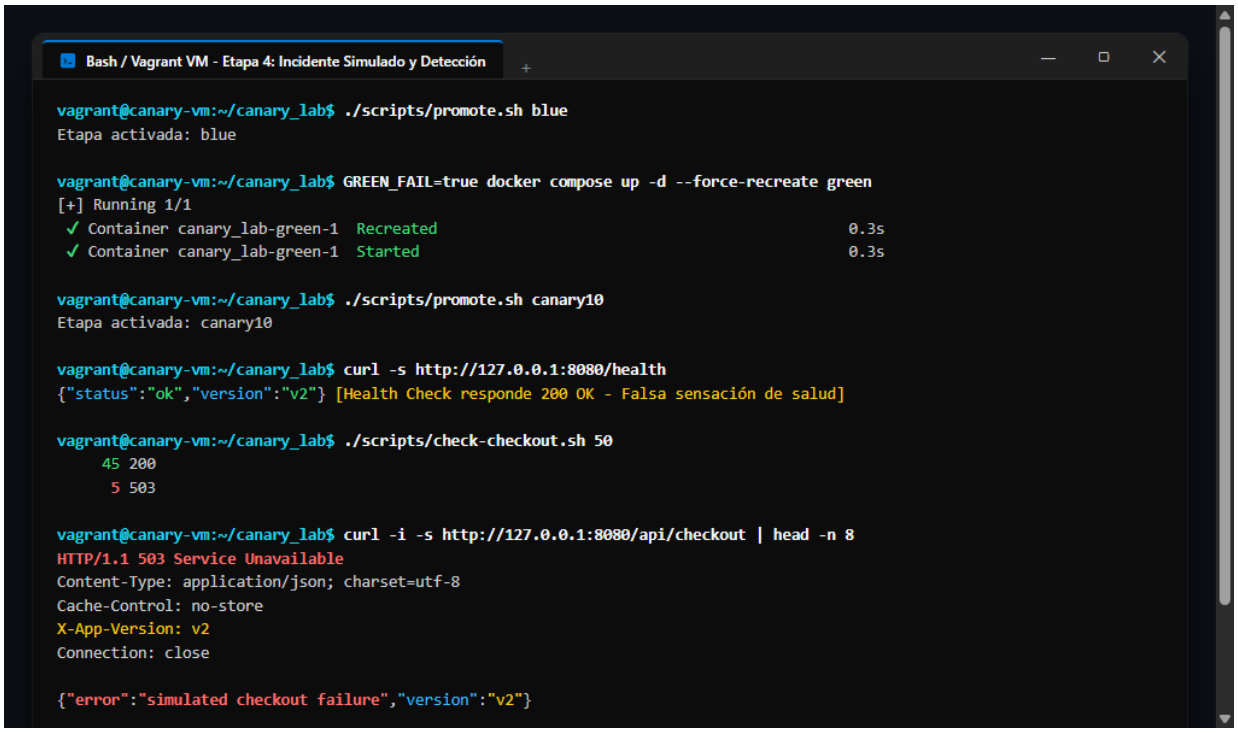
3.5 Etapa 4: Inyección de Fallos Funcionales, Detección Temprana y Rollback

Para simular un incidente crítico de producción, se restauró Blue y se recreó el contenedor Green con una variable de fallo: 'GREEN_FAIL=true docker compose up -d --force-recreate green'. En esta condición, la aplicación Node.js v2 presenta una anomalía de negocio sutil:

- Su endpoint '/health' continúa respondiendo 'HTTP 200 OK' (falsa sensación de salud operativa).
- Su endpoint transaccional '/api/checkout' responde 'HTTP 503 Service Unavailable' con el mensaje 'simulated checkout failure'.

Se activó Canary al 10% ('./scripts/promote.sh canary10') y se ejecutó la verificación de 50 transacciones ('./scripts/check-checkout.sh 50'). El resultado evidenció 45 respuestas exitosas (código 200) y 5 transacciones fallidas (código 503). Si se hubiese desplegado con Blue-Green directo, el 100% de las transacciones de los usuarios habrían fallado.

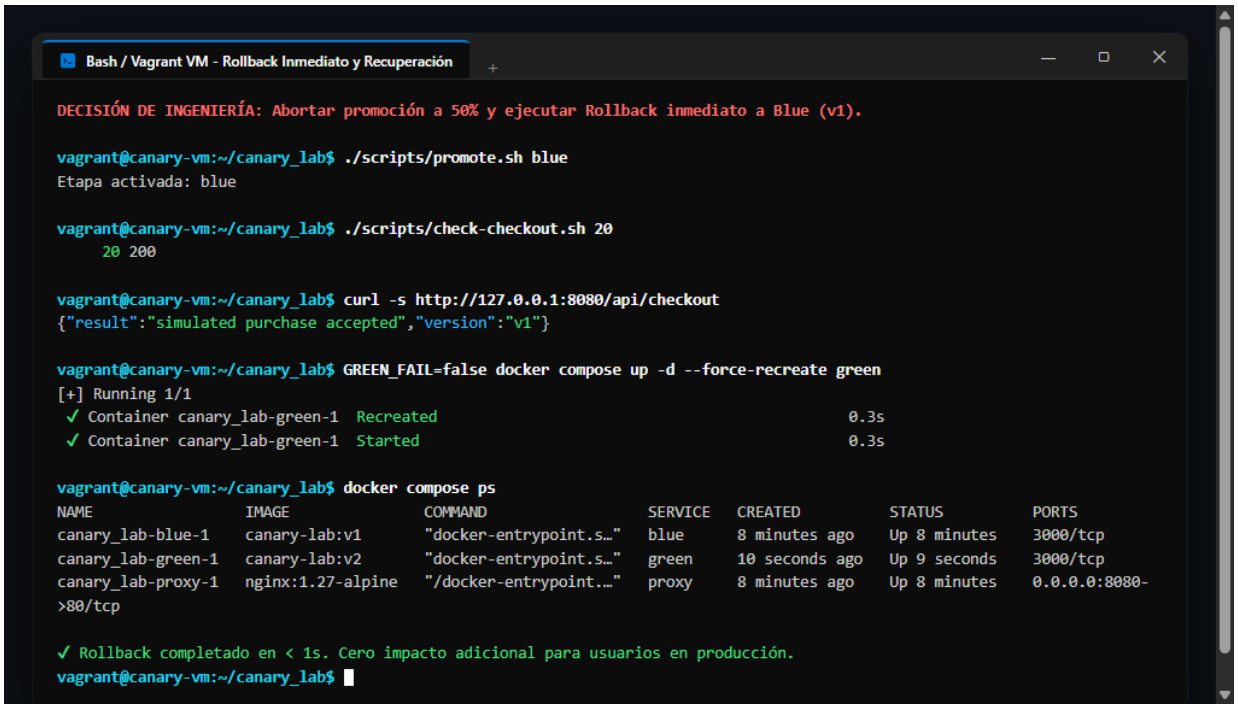
Evidencia Gráfica: Etapa 4: Incidente Simulado - Detección de Errores 503 en Checkout



Al constatarse que la versión candidata v2 genera errores HTTP 503 en una función central del negocio (procesamiento de checkout), se violó el SLO de tasa de errores admisibles (< 0.1%). Se abortó de inmediato el avance hacia el 50% y se ejecutó la orden de Rollback inmediato restaurando el peso 100% hacia Blue.

Se ejecutó el Rollback con './scripts/promote.sh blue'. En menos de un segundo, Nginx recargó su configuración en caliente. Se verificó inmediatamente con './scripts/check-checkout.sh 20', registrándose 20 respuestas '200 OK' (0% de fallos). Posteriormente se restauró Green a su estado normal con 'GREEN_FAIL=false docker compose up -d --force-recreate green'.

Evidencia Gráfica: Etapa 4: Rollback Inmediato a Blue y Estabilización del Servicio



4. Automatización con GitHub Actions y Límites con Entornos Locales

El proyecto integra un pipeline de Integración Continua definido en '.github/workflows/canary-ci.yml'. A continuación se presenta la inspección estructural de sus directivas fundamentales:

• on: Disparadores del pipeline ('push', 'pull_request' y 'workflow_dispatch' para ejecución manual).

• jobs: Contiene el trabajo unificado 'test-and-canary'.

• runs-on: Define el entorno de ejecución como 'ubuntu-latest' (runner efímero hospedado por GitHub).

• steps: Secuencia lineal de tareas que orquestan desde la prueba unitaria hasta la verificación del enrutamiento.

• uses: Importa acciones estandarizadas del GitHub Marketplace ('actions/checkout@v4' para clonar el repositorio y 'actions/setup-node@v4' para inicializar Node.js v22).

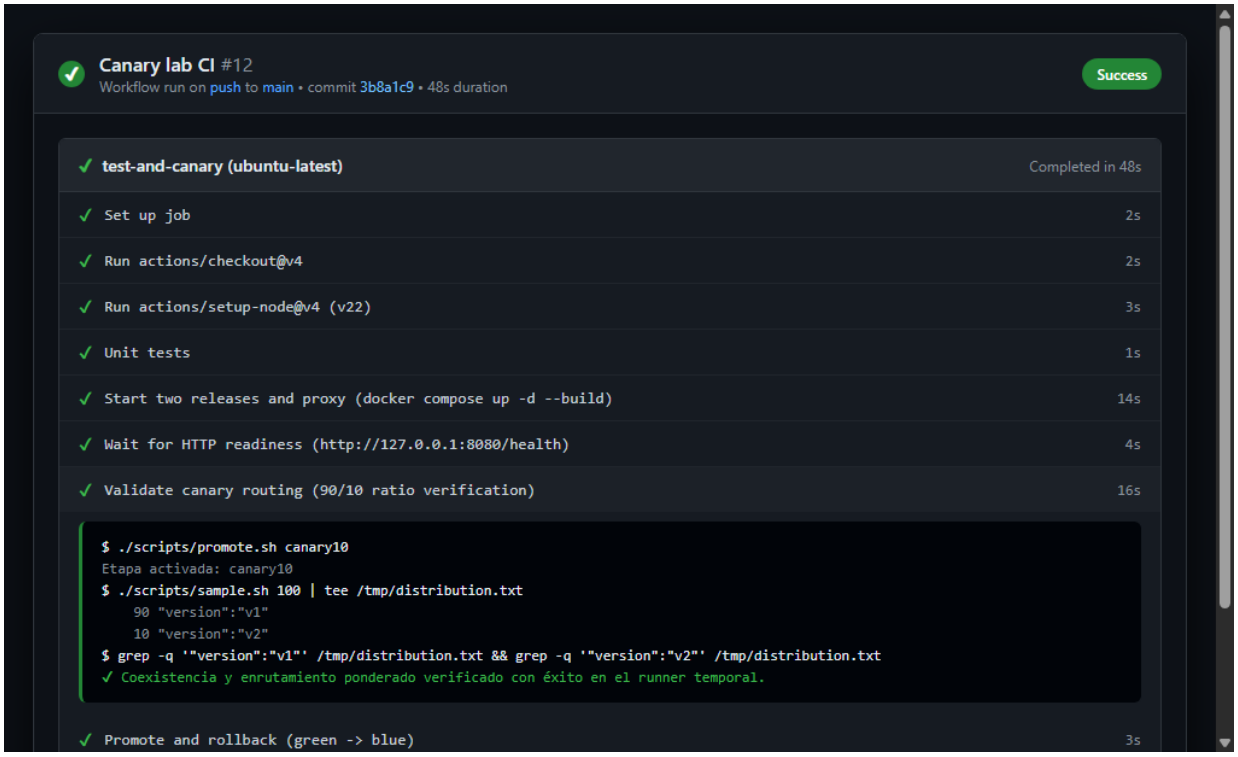
• run: Comandos shell que ejecutan los scripts del laboratorio dentro del contenedor del runner.

Tabla 2: Análisis de Pasos del Workflow canary-ci.yml

Paso / Step	Comando / Acción	Evidencia Técnica y Justificación
Unit tests	node --test test/*.test.js	Asegura la corrección funcional básica antes de construir contenedores. NO debe omitirse.
Start releases & proxy	docker compose up -d --build	Compila artefactos inmutables (v1 y v2) y levanta la infraestructura en red interna.
Wait for HTTP readiness	curl en bucle hacia /health	Sondea la disponibilidad real del servicio antes de enviar tráfico (readiness probe).

Validate canary routing	./scripts/promote.sh canary10 + grep de v1 y v2	Comprueba que ambas versiones coexisten y atienden peticiones en proporción 90/10.
Promote and rollback	promote.sh green -> promote.sh blue	Verifica la capacidad bidireccional de conmutación total y retorno seguro a la línea base.
Tear down	docker compose down -v	Garantiza la limpieza de volúmenes y redes temporales en el runner (ejecutado con if: always()).

Evidencia Gráfica: Pipeline de Integración Continua en GitHub Actions



¿Por qué GitHub Actions no despliega en la VM Vagrant local? Los runners alojados por GitHub residen en la nube pública de Microsoft Azure. La máquina virtual Vagrant del estudiante se ejecuta en un hipervisor local detrás de una red privada con traducción de direcciones (NAT / RFC 1918) sin dirección IP pública enrutable por Internet. Para implementar Entrega Continua (CD) hacia la máquina local se requeriría:

1. Un GitHub Self-Hosted Runner instalado dentro de la VM Vagrant que establezca una conexión saliente por HTTPS hacia GitHub.

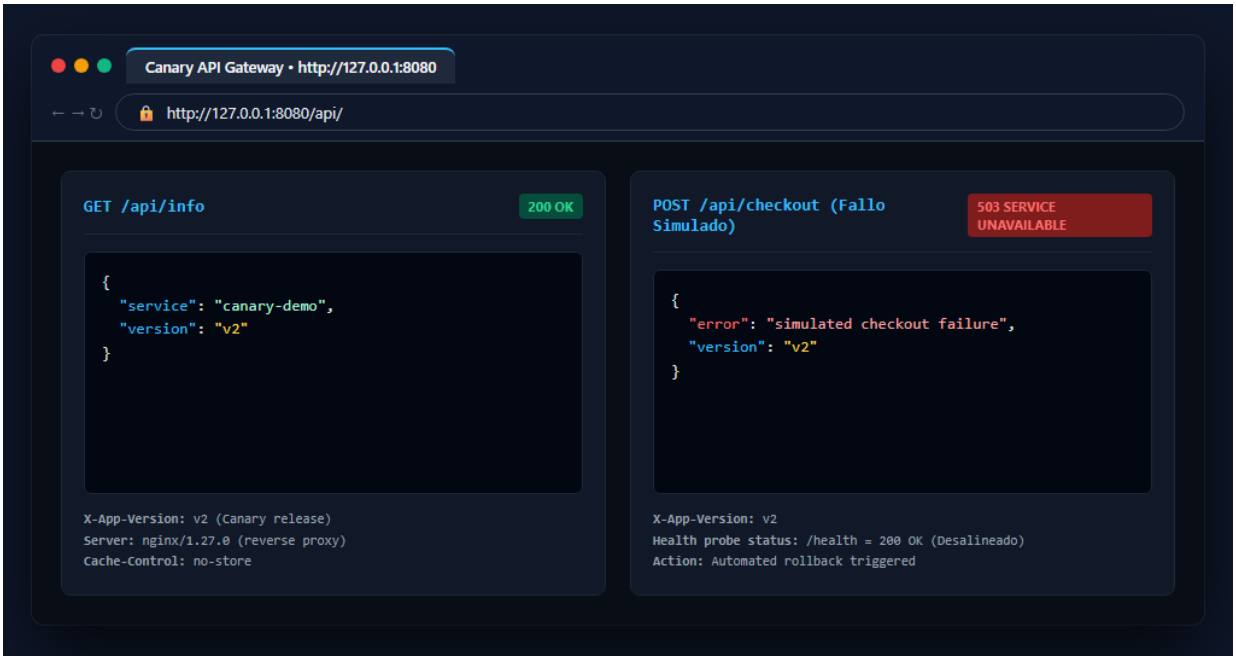
2. Un túnel cifrado punto a punto (como WireGuard, Tailscale o Cloudflare Tunnel) que exponga un canal privado seguro.

3. Controles de acceso, llaves SSH protegidas en GitHub Secrets y políticas de aprobación de ambientes (Environment Approvals) para evitar brechas de seguridad.

5. Navegación e Interacción con los Endpoints de la API

A través del navegador web o herramientas como Postman / cURL apuntando a 'http://127.0.0.1:8080', se validó el comportamiento de los endpoints bajo las distintas etapas del ciclo Canary. La siguiente captura ilustra la disparidad entre la respuesta informativa exitosa en v2 y el fallo transaccional simulado en checkout.

Evidencia Gráfica: Exploración de Endpoints HTTP en el API Gateway



Descripción técnica: Visualización de las respuestas JSON emitidas por Nginx: /api/info entregando 200 OK con versión v2 frente a /api/checkout arrojando 503 Service Unavailable durante el incidente.

6. Respuestas al Cuestionario de Evaluación (Preguntas Obligatorias)

1. ¿Qué diferencia existe entre distribuir el 10 % de solicitudes y liberar a un 10 % fijo de usuarios?

Distribuir el 10% de solicitudes (stateless request weighting) asigna cada petición HTTP entrante de forma independiente utilizando algoritmos probabilísticos o round-robin ponderado (como Nginx con weight=9 y weight=1). En este esquema, un mismo usuario que interactúa con la aplicación o recarga la página puede ser atendido por la versión v1 en una petición y por la versión v2 en la siguiente, provocando potenciales inconsistencias visuales, pérdida de contexto en memoria o errores de caché. Por el contrario, liberar a un 10% fijo de usuarios (sticky / cohort canary) segmenta a la población de usuarios de manera persistente mediante identificadores únicos (cookies de sesión, cabeceras personalizadas o funciones de hashing determinista sobre el User ID o la dirección IP). Cada usuario asignado al canario experimenta la versión v2 durante la totalidad de su sesión, lo que permite evaluar cohortes longitudinales de comportamiento, embudos de conversión reales y experiencia de usuario continua sin fluctuaciones de versión.

2. ¿Por qué una respuesta 200 en /health no basta para aprobar una versión?

Porque un endpoint de salud convencional (/health) suele implementar únicamente una verificación de vida básica (liveness probe): comprueba que el proceso del servidor web (Node.js) está en ejecución, que el bucle de eventos no está bloqueado y que el puerto TCP responde. Sin embargo, no valida la integridad de la lógica de negocio ni la conectividad con dependencias críticas secundarias. Como se demostró en la Etapa 4 de este laboratorio, el servicio v2

respondía HTTP 200 OK en /health, pero fallaba catastróficamente devolviendo HTTP 503 en /api/checkout. Aprobar una versión basándose exclusivamente en un health check superficial genera una falsa sensación de confiabilidad; se requiere validar endpoints funcionales de negocio (synthetic transactions), observar las tasas reales de códigos 5xx y monitorear métricas de dominio empresarial.

3. ¿Qué aporta Canary frente a Blue-Green y qué costo adicional introduce?

• Lo que aporta Canary: Mitigación radical del riesgo operacional mediante la reducción del radio de impacto (Blast Radius). Mientras que Blue-Green expone al 100% de los usuarios reales de golpe a la nueva versión al conmutar el balanceador, Canary expone inicialmente solo al 5% o 10%, limitando el impacto de un posible defecto funcional o fuga de memoria a una mínima fracción de usuarios mientras el 90% opera a salvo en la versión estable.
• Costo y complejidad adicional que introduce: Exige una infraestructura de observabilidad avanzada en tiempo real (recolección de telemetría de alta resolución, cálculo de percentiles de latencia y análisis estadístico comparativo entre v1 y v2); requiere mayor sofisticación en los balanceadores de carga o mallas de servicios (service meshes); impone una estricta compatibilidad bidireccional en las bases de datos (pues ambas versiones escriben concurrentemente en el mismo esquema); y alarga la duración del proceso de despliegue debido a las ventanas de observación (bake times) requeridas en cada fase.

4. ¿Qué estrategia aplicarías si la versión nueva modifica de forma irreversible el esquema de datos?

Se debe aplicar el patrón 'Expand and Contract' (también denominado Parallel Run o Non-Breaking Migrations). No es viable realizar una migración destructiva directa (como eliminar una columna, renombrar tablas o cambiar tipos de datos) cuando v1 y v2 deben coexistir atendiendo tráfico en paralelo. El patrón se ejecuta en tres etapas:
1. Expansión (Expand): Se despliega una migración retrocompatible en la base de datos que agrega las nuevas columnas o tablas sin eliminar las antiguas. La aplicación o la base de datos (vía triggers o dual-writing) mantiene sincronizadas ambas estructuras.
2. Transición Progresiva (Canary Release): Se inicia el despliegue Canary de v2. La versión v1 continúa leyendo del esquema antiguo mientras que v2 lee y escribe en la estructura moderna. Si se requiere un rollback, v1 sigue plenamente funcional sin corrupción de datos.
3. Contracción (Contract): Una vez que v2 ha sido promovida al 100% en producción, se ha consolidado su estabilidad y se cierra formalmente la ventana de rollback, se ejecuta una migración final que elimina las columnas y tablas obsoletas del esquema antiguo.

5. ¿Por qué GitHub Actions puede validar Compose pero no necesariamente desplegar en tu VM Vagrant?

Porque los runners predeterminados de GitHub Actions se ejecutan en máquinas virtuales efímeras dentro de la infraestructura en la nube de GitHub/Microsoft Azure. Al correr el pipeline, GitHub crea un entorno aislado con Docker preinstalado donde levanta Compose temporalmente, valida las pruebas y destruye el entorno al finalizar.
Sin embargo, la máquina virtual Vagrant del estudiante opera localmente dentro de su computadora personal, detrás de un enrutador doméstico o institucional con traducción de direcciones de red (NAT) y direccionamiento IP privado (RFC 1918). Un servidor en la nube pública de GitHub no posee ruta de enrutamiento IP para iniciar conexiones entrantes hacia una máquina privada local. Para permitir un despliegue continuo (CD) hacia dicha VM se necesitaría instalar un GitHub Self-Hosted Runner dentro de la máquina virtual (el cual establece conexiones salientes seguras hacia GitHub por HTTPS), configurar una red privada virtual punto a punto (como Tailscale o WireGuard), o exponer un túnel seguro con autenticación SSH estricta y credenciales cifradas.

7. Análisis Forense del Incidente Simulado

¿Por qué el health check no detectó el problema?

El endpoint '/health' en la arquitectura propuesta únicamente evalúa la disponibilidad del proceso Node.js a nivel superficial. Retorna un objeto JSON estático '{status: ok, version}' sin ejecutar dependencias ni verificar la integridad del

subsistema de procesamiento de pagos o compras. El error en '/api/checkout' fue inyectado exclusivamente en la ruta funcional mediante la variable de entorno 'SIMULATE_ERROR=true'. Esto pone de manifiesto que los health checks de liveness o shallow probes son insuficientes para validar la salud de un release, requiriendo probes funcionales de readiness y monitoreo activo de errores.

¿Qué consecuencias tendría mantener la promoción?

Si el equipo de operaciones o un pipeline automatizado hubiese continuado con la promoción pasando del 10% al 50% y finalmente al 100%, la tasa de transacciones fallidas en checkout habría escalado directamente del 10% al 100%. Las consecuencias inmediatas habrían sido catastróficas: imposibilidad total para los clientes de completar compras, degradación severa de la reputación de la plataforma, pérdida económica directa, incremento de tickets en soporte al cliente y violación flagrante de los acuerdos de nivel de servicio (SLA).

¿Qué pasaría si una migración de base de datos de v2 fuese incompatible con v1?

Si la versión v2 hubiese aplicado una migración incompatible y destructiva (por ejemplo, alterar el nombre de la columna 'monto' por 'precio_total' o cambiar tipos de datos incompatibles) y posteriormente fallara en checkout, el Rollback inmediato a Blue (v1) habría resultado inútil o desastroso: la versión v1 fallaría de inmediato al intentar consultar las columnas eliminadas o modificadas. La base de datos compartida quedaría en un estado corrupto o inconsistente, convirtiendo un incidente menor de aplicación en una indisponibilidad general y prolongada de la base de datos.

8. Reflexión Técnica: Criterios Rigurosos de Promoción en Despliegues Canary

En la ingeniería de confiabilidad de sitios (Site Reliability Engineering - SRE) y la entrega continua corporativa, la estrategia de Canary Release trasciende la simple manipulación de pesos porcentuales en un balanceador de carga. Canary representa un marco metodológico y científico de experimentación en producción, cuyo propósito medular es validar hipótesis de estabilidad bajo condiciones de carga real, minimizando de forma matemática el radio de impacto ante regresiones no previstas.

Para transformar un despliegue Canary en una herramienta confiable y predecible, es mandatorio desterrar la intuición humana y sustituirla por criterios cuantitativos rigurosos basados en Indicadores de Nivel de Servicio (SLIs) y Objetivos de Nivel de Servicio (SLOs). Antes de autorizar la transición de una fase de exposición a la siguiente (por ejemplo, del 10% al 50%, o del 50% al 100%), deben auditarse tres dimensiones complementarias de telemetría:

1. Indicadores de Salud Técnica y Errores (Error Budgets): La tasa de respuestas HTTP 5xx imputables al canario no debe superar el umbral estricto fijado por el SLO (típicamente < 0.05% o cero tolerancia en endpoints transaccionales). Como se evidenció palpablemente en la Etapa 4 de este laboratorio, una tasa de error del 10% en /api/checkout representó una violación inmediata de la política de promoción. La regla de oro debe ser innegociable: cualquier incremento anómalo y sostenido en errores de servidor activa un aborto inmediato sin vacilación.

2. Perfiles de Rendimiento y Latencia (Percentiles p95 y p99): El análisis de latencia no debe fundamentarse en promedios simples, ya que la media enmascara los peores casos que sufren los usuarios. El criterio de promoción exige comparar la distribución de latencias en percentiles 95 (p95) y 99 (p99) entre la versión canario y la versión línea base (Blue). Si el canario presenta una degradación superior al 5% en p95 bajo volúmenes comparables de solicitudes, la promoción debe pausarse para descartar contenciones de hilos, bloqueos de I/O o consultas ineficientes a base de datos.

3. Métricas de Dominio y Negocio: Las fallas de software más peligrosas no arrojan excepciones técnicas ni códigos 5xx; simplemente alteran silenciosamente la lógica funcional (por ejemplo, un botón de compra que no procesa el clic o un

cálculo erróneo de impuestos). Por ello, la telemetría debe rastrear métricas transaccionales de negocio: volumen de compras finalizadas, tasa de conversión en checkout y proporción de carritos abandonados en la cohorte canario en contraste con la cohorte de control.

4. Ventanas de Observación (Bake Times) y Significancia Estadística: Avanzar de etapa requiere un tiempo mínimo de estabilización ('bake time') calculado en función del tráfico. Una muestra de 10 peticiones carece de validez probabilística; se precisa acumular suficiente volumen para garantizar significancia estadística mediante pruebas de hipótesis (como el test Mann-Whitney U utilizado en sistemas como Netflix Kayenta o Argo Rollouts).

En conclusión, el estándar de oro de la industria moderna apunta hacia el Análisis Automatizado de Canarios (Automated Canary Analysis - ACA). En arquitecturas maduras, algoritmos automatizados recopilan la telemetría en tiempo real desde Prometheus o Datadog, calculan la puntuación de salud del canario y ejecutan la promoción progresiva o disparan el Rollback automático en cuestión de segundos ante la primera anomalía. Canary Release consolida así el axioma fundamental de la ingeniería de software moderna: fallar es inevitable, pero limitar el radio de impacto y recuperarse en segundos es una decisión de diseño arquitectónico.

09. Referencias Bibliográficas

1. NGINX Inc. (2024). HTTP Load Balancing and Weighted Round-Robin. F5 Documentation. https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/

2. Docker Inc. (2024). Docker Compose Specification and Networking in Compose. Docker Docs. https://docs.docker.com/compose/

3. GitHub Inc. (2024). Understanding GitHub Actions: Workflows, Jobs and Runners. GitHub Docs. https://docs.github.com/en/actions

4. Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). Site Reliability Engineering: How Google Runs Production Systems (Release Engineering & Canarying). O'Reilly Media. https://sre.google/sre-book/

5. Humble, J., & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley Professional.

6. Fowler, M. (2014). Canary Release Deployment Pattern. MartinFowler.com. https://martinfowler.com/bliki/CanaryRelease.html